

Relatório de Implementação — Aula 2

Banco e base da aplicação

Campo	Conteúdo
Disciplina	Banco de Dados II
Trabalho	Plataforma de Gerenciamento de Backup
Equipe	André Júnior, Davi Zappellini, Gabriel de Bona, José Henrique Pereira
Repositório	https://github.com/Andreven04/trabalho_banco2
Data da entrega	23/09/2026

1. Resumo (planejado × concluído)

Planejado para esta aula: implementar a estrutura do banco (DDL), a carga inicial de massa de dados, e a conexão da aplicação ao banco, além das telas iniciais e das instruções de execução.

Concluído nesta aula:

- Banco hospedado no Supabase (PostgreSQL 17.6), com dois schemas: loja (domínio) e plataforma (metadados da ferramenta de backup).
- Estrutura (DDL) das 14 tabelas criada e executada com sucesso.
- Carga de massa do domínio concluída, com volumes alinhados ao Documento de Visão.
- Backend Fastify inicializado, conectando ao Supabase, com o endpoint de teste de conexão.
- Scripts versionados no repositório e instruções de execução (README) publicadas.

2. Ambiente e stack

Camada	Tecnologia
Hospedagem do banco	Supabase (PostgreSQL 17.6)
Acesso ao banco	Cliente psql via Session pooler (IPv4), porta 5432, SSL
Backend / API	Node.js + Fastify (TypeScript), driver pg
Frontend	Angular (em desenvolvimento)

3. Modelo de dados implementado

O banco foi separado em dois schemas, conforme o documento de visão:

Schema loja (domínio — loja online):

- categoria, produto, cliente, pedido, pedido_item, pagamento, estoque_movimento.

Schema plataforma (metadados da plataforma de backup):

- banco_alvo, config_backup, execucao, execucao_etapa, manutencao_registro, backup_arquivo, log_evento.

Observação: o Supabase fornece um único banco por projeto; por isso a separação domínio × metadados foi implementada por schemas, e não por bancos distintos.

4. Scripts de criação e carga

Todos versionados no repositório, reproduzíveis em outro ambiente:

Arquivo	Conteúdo
sql/dominio/01_estrutura.sql	DDL das 7 tabelas do domínio (schema loja)
sql/plataforma/01_estrutura.sql	DDL das 7 tabelas de metadados (schema plataforma)
sql/dominio/02_carga.sql	Carga de massa do domínio (generate_series)
sql/dominio/03_churn.sql	Geração de dead tuples (material da Aula 3)

5. Evidências do banco populado

Contagem de registros por tabela após a carga (saída real da execução):

Tabela	Registros
categoria	4
produto	2.000
cliente	5.000
pedido	50.000
pedido_item	150.000
pagamento	25.000
estoque_movimento	300.000

Print da execução no psql (consulta de contagens):

```
postgres=> SELECT 'categoria' AS tabela, COUNT(*) FROM loja.categoria
postgres-> UNION ALL SELECT 'produto', COUNT(*) FROM loja.produto
postgres-> UNION ALL SELECT 'cliente', COUNT(*) FROM loja.cliente
postgres-> UNION ALL SELECT 'pedido', COUNT(*) FROM loja.pedido
postgres-> UNION ALL SELECT 'pedido_item', COUNT(*) FROM loja.pedido_item
postgres-> UNION ALL SELECT 'pagamento', COUNT(*) FROM loja.pagamento
postgres-> UNION ALL SELECT 'estoque_movimento', COUNT(*) FROM loja.estoque_movimento
postgres-> ORDER BY tabela;
tabela | count
-----+-----
categoria | 4
cliente | 5000
estoque_movimento | 300000
pagamento | 25000
pedido | 50000
pedido_item | 150000
produto | 2000
(7 linhas)
```

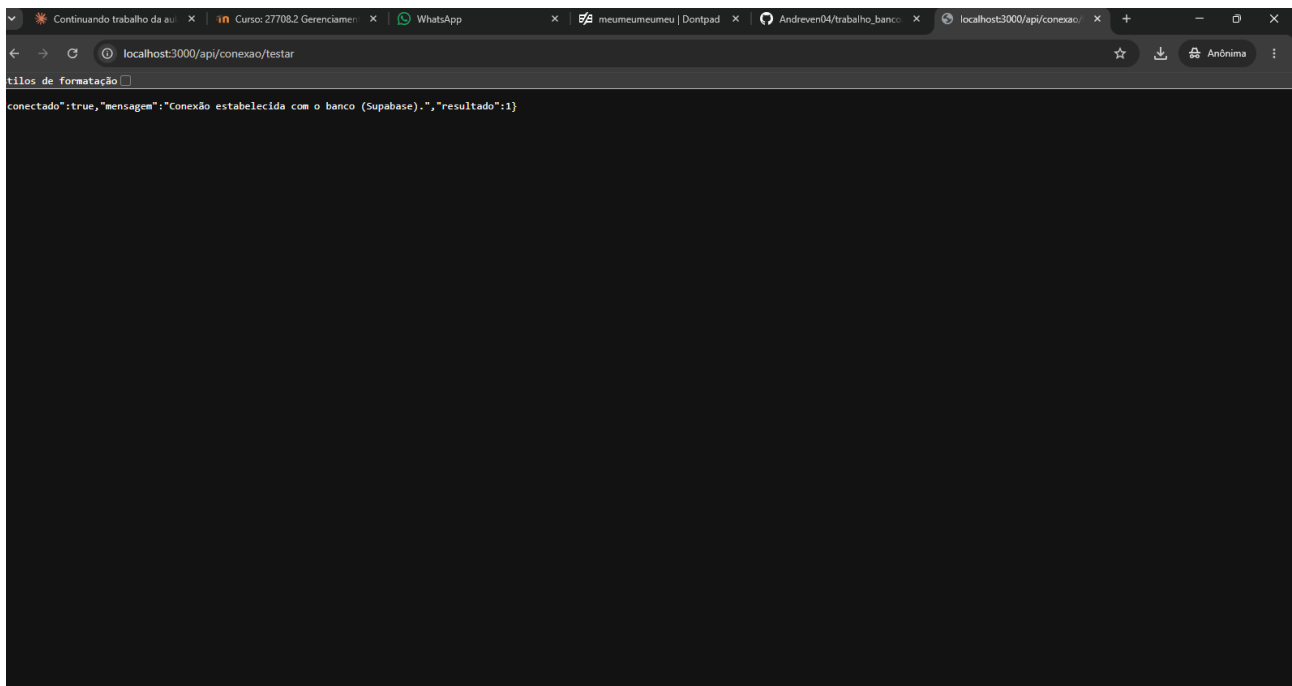
6. Conexão pela aplicação

O backend Fastify conecta ao Supabase por um pool de conexões (driver pg) e expõe o endpoint GET /api/conexao/testar, que executa uma consulta simples (SELECT 1) e retorna o status da conexão.

Resposta esperada do endpoint:

```
{ "conectado": true, "mensagem": "Conexão estabelecida com o banco (Supabase).",
  "resultado": 1 }
```

Print do endpoint respondendo:



7. Telas iniciais

A tela de configuração e conexão (Angular), com o botão "Testar conexão" consumindo o endpoint acima, ficou como pendência desta aula e será concluída na próxima etapa (ver seção 10).

8. Instruções para executar o projeto

Passos resumidos (detalhe completo no README.md do repositório):

- Pré-requisitos: Node.js 20+, Git e um cliente PostgreSQL (psql).
- Banco: criar os schemas loja e plataforma e rodar os scripts na ordem — 01_estrutura (domínio), 01_estrutura (plataforma), 02_carga (domínio).
- Backend: na pasta backend, rodar npm install, configurar o arquivo .env na raiz com a string de conexão do Supabase, e iniciar com npm run dev.
- Testar: acessar <http://localhost:3000/api/conexao/testar> e verificar a resposta conectado:true.

9. Contribuições de cada integrante

(Ajustar conforme o que cada um efetivamente executou.)

Integrante	Contribuição na Aula 2
Gabriel de Bona	Modelagem do domínio; scripts de estrutura, carga e churn; massa de dados e evidências.
José Henrique Pereira	Modelagem dos metadados da plataforma; scripts de estrutura do schema plataforma.
André Júnior	Setup do repositório e ambiente; backend Fastify e endpoint de conexão; relatório.
Davi Zappellini	Backend base e validação da conexão ao Supabase; instruções de execução.

10. Pendências, problemas encontrados e plano para a Aula 3

Problemas encontrados e como foram resolvidos:

- Supabase fornece um único banco por projeto — resolvido usando dois schemas (loja e plataforma) no mesmo banco.
- A conexão direta do Supabase é IPv6 — resolvido usando o Session pooler (IPv4) na porta 5432, com SSL.
- Aviso de página de código do Windows no psql — os nomes de categoria foram carregados sem acentos para evitar caracteres corrompidos.

Pendências:

- Concluir a tela inicial do frontend Angular (configuração + botão Testar conexão consumindo o endpoint).

Plano para a Aula 3 (Manutenção e backup):

- Implementar a decisão automática de manutenção pelas faixas de dias (< 30, 30–60, > 60 e sem histórico).
- Executar VACUUM / VACUUM FULL ANALYZE e comprovar o efeito nos dead tuples (usando o script de churn).
- Gerar o backup com pg_dump e registrar as evidências dos cenários exigidos.

Repositório: https://github.com/Andreven04/trabalho_banco2